

Day 83: SQL Fundamentals — `SELECT` and `WHERE`

1. Day Number

Day 83

2. Topic Name

Querying Database Data with SQL

Today you will learn how to **retrieve information from a database table** using simple SQL queries.

The main commands are:

```
SELECT
FROM
WHERE
ORDER BY
```

3. Connection

Yesterday, you learned that a database can contain tables made of:

```
rows
columns
primary keys
```

For example:

```
products
-----
product_id | name      | price
-----
1           | Keyboard | 1200
2           | Mouse   | 600
3           | Monitor  | 9000
```

Today you will learn how to ask the database questions such as:

```
Show me all products.

Show me products costing more than 1000.

Show me products from cheapest to most expensive.
```

SQL lets us express these questions as queries.

4. Important Topics

SELECT

SELECT tells the database **which columns you want to see**.

Example:

```
SELECT name
```

means:

```
Show me the name column.
```

You will also often see:

```
SELECT *
```

The `*` means:

```
all columns
```

FROM

FROM tells SQL **which table to read**.

Example:

```
FROM products
```

means:

```
Get the data from the products table.
```

WHERE

WHERE filters the rows.

For example:

```
WHERE price > 1000
```

means:

```
Only keep products whose price is greater than 1000.
```

Comparison operators

Some common SQL comparison operators are:

Operator	Meaning
=	equal to
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
<> or !=	not equal to

For example:

```
price > 1000
```

and:

```
price >= 1000
```

are slightly different.

The second one also includes products priced **exactly 1000**.

ORDER BY

ORDER BY sorts the result.

Example idea:

```
ORDER BY price
```

usually sorts from lower values to higher values.

You may also encounter:

```
ORDER BY price DESC
```

where DESC means descending:

```
highest → lowest
```

5. Foundational Notes

SQL stands for:

Structured Query Language

It is commonly used to communicate with relational databases.

You can think of SQL as a language for asking questions about tables.

Suppose you have:

product_id	name	price
1	Keyboard	1200
2	Mouse	600
3	Monitor	9000
4	Webcam	1800

A SQL query might ask:

From the products table,
show me products where
price is greater than 1000.

The database examines the rows and returns only the ones that satisfy the condition.

6. A SQL Query in Plain English

Consider this query structure:

```
SELECT ...  
FROM ...  
WHERE ...  
ORDER BY ...;
```

Read it in plain English as:

```
SELECT  
→ What information do I want?  
  
FROM  
→ Which table should I look inside?  
  
WHERE  
→ Which rows should I keep?  
  
ORDER BY  
→ How should I sort the result?
```

For example:

```
SELECT name, price
FROM products
WHERE price > 1000;
```

means:

Show the name and price
from the products table
for products costing more than 1000.

The semicolon:

;

marks the end of the SQL statement.

7. Easy Example

Suppose our table is:

products

product_id	name	price
1	Keyboard	1200
2	Mouse	600
3	Monitor	9000
4	Webcam	1800
5	Cable	300

Suppose you only wanted to see product names.

You could think:

What column?
→ name

What table?
→ products

A simple SQL query would follow this pattern:

```
SELECT name
FROM products;
```

Expected result:

name
Keyboard
Mouse
Monitor
Webcam
Cable

This is a basic `SELECT` query without filtering.

8. Problem Statement

Use this simple `products` table:

product_id	name	price
1	Keyboard	1200
2	Mouse	600
3	Monitor	9000
4	Webcam	1800
5	Cable	300
6	Headphones	1200

Write SQL queries that conceptually perform these tasks:

1. Show **all products**.
2. Show only products whose price is **greater than 1000**.
3. Show all products ordered by **price from lowest to highest**.

Try to write each query yourself before looking at the hint.

9. Concepts Used

This exercise uses:

```
database table
rows
columns
SQL query
SELECT
FROM
WHERE
comparison operators
```

```
ORDER BY  
ascending order  
filtering
```

The basic mental model is:

```
Table  
↓  
SELECT columns  
↓  
WHERE condition  
↓  
ORDER BY column  
↓  
Result
```

10. Thought Process

Suppose the question says:

```
| Show products costing more than 1000.
```

Think through it in this order.

Step 1: Which table contains the data?

```
products
```

Step 2: Which columns do I want?

Perhaps all product information:

```
product_id  
name  
price
```

Or simply:

```
all columns
```

Step 3: Is there a condition?

Yes:

```
price greater than 1000
```

In SQL-style thinking:

```
price > 1000
```

Then combine those ideas into the correct SQL structure.

11. Beginner-Friendly Pseudocode Before SQL

Task 1: Show every product

```
START  
  
look inside the products table  
  
select every column  
  
return every row  
  
END
```

SQL pattern:

```
SELECT ...  
FROM ...;
```

Task 2: Find expensive products

```
START  
  
look inside the products table  
  
select the required columns  
  
check each product price  
  
if price is greater than 1000  
    include that row  
  
return matching rows  
  
END
```

SQL pattern:

```
SELECT ...  
FROM ...  
WHERE ...;
```

Task 3: Sort products by price

```
START

look inside the products table

select the required columns

sort using the price column
from smallest price to largest price

return the sorted rows

END
```

SQL pattern:

```
SELECT ...
FROM ...
ORDER BY ...;
```

12. Suggested Solving Approach: SQL

For each question, identify four things:

1. Which table?
2. Which columns?
3. Do I need a condition?
4. Do I need sorting?

For this exercise:

```
Table
→ products
```

Then decide whether you need:

```
SELECT
```

only, or:

```
SELECT + WHERE
```

or:

```
SELECT + ORDER BY
```

You do not need subqueries, joins, or advanced SQL for today's exercise.

13. Easy Edge Cases

No Matching Rows

Suppose you ask for:

```
price > 50000
```

but your most expensive product costs only `9000`.

The database may return:

```
0 rows
```

That does **not** necessarily mean the query failed.

It simply means:

```
No row satisfied the condition.
```

Equal Values

Our table contains:

```
Keyboard    → 1200  
Headphones  → 1200
```

If you filter using:

```
price = 1200
```

both rows should match.

If you sort by price, equal-price products may appear beside each other.

This is completely normal.

Also remember:

```
price > 1200
```

does **not** include `1200`.

But:

```
price >= 1200
```

does.

14. Expected Result Table

For the condition:

```
price > 1000
```

you should expect rows conceptually similar to:

product_id	name	price
1	Keyboard	1200
3	Monitor	9000
4	Webcam	1800
6	Headphones	1200

Notice that:

```
Mouse → 600  
Cable → 300
```

are not included because they do not satisfy:

```
price > 1000
```

For the sorting task, the result should begin with the cheapest product and end with the most expensive one.

15. Hint Only

For **show all products**, remember that:

```
*
```

means:

```
all columns
```

For **products above 1000**, your query needs this structure:

```
SELECT ...  
FROM products  
WHERE price ... 1000;
```

Think carefully about which comparison operator belongs in the blank.

For **sorting by price**, think:

```
SELECT ...  
FROM products  
ORDER BY ...;
```

Your goal for Day 83 is to understand this flow:

```
SELECT → what should I display?  
  
FROM → where is the data?  
  
WHERE → which rows should remain?  
  
ORDER BY → how should the result be sorted?
```

Try writing the **three complete queries yourself** before moving on to more SQL commands.